

◆ CS EDUCATION ◆

🔍

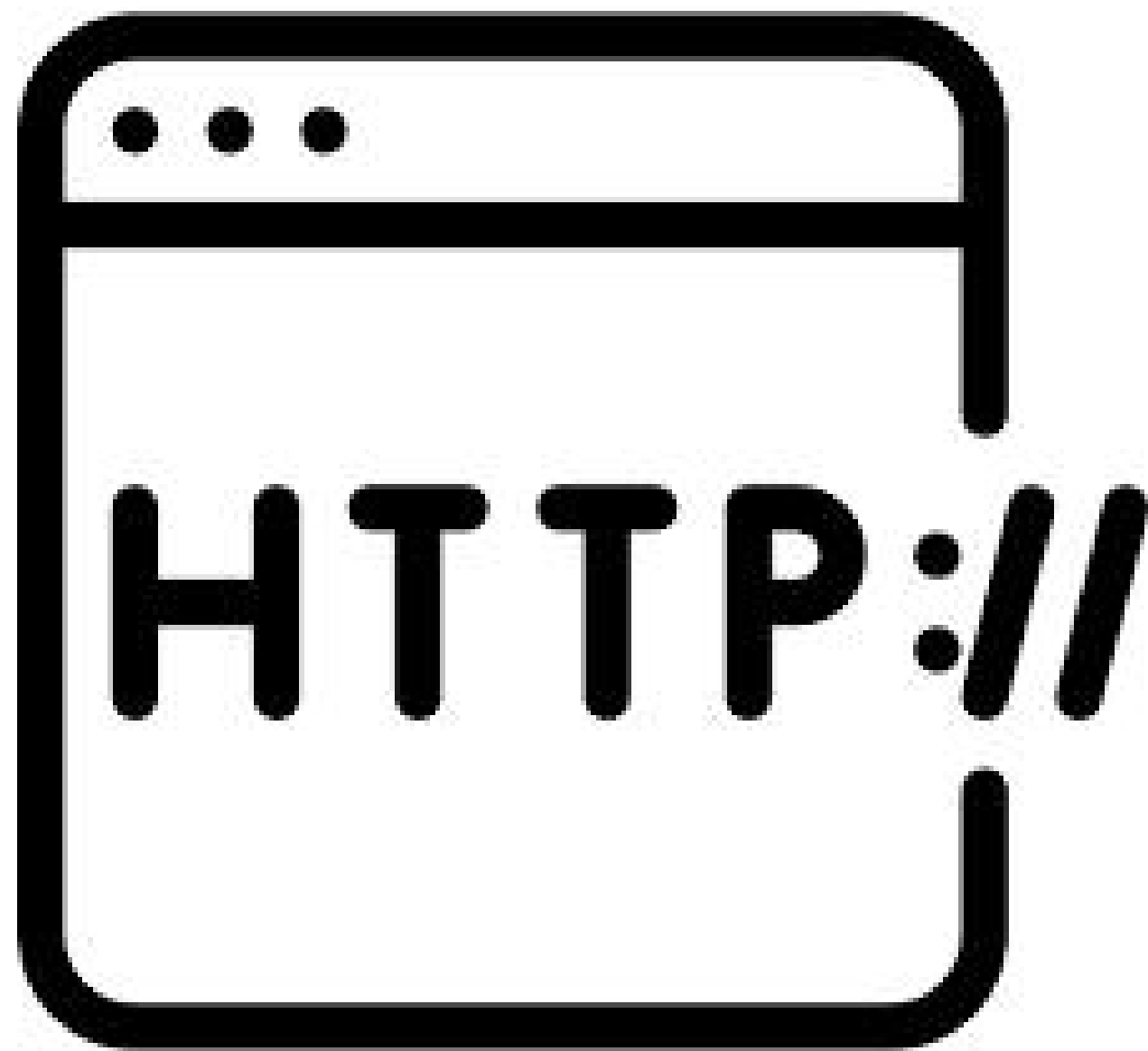
WebSocket

26.04.03
강철웅

교육 목표

1. WebSocket이 왜 필요한가?
2. SSE vs WebSocket
3. STOMP

🌐 WebSocket이 왜 필요한가?

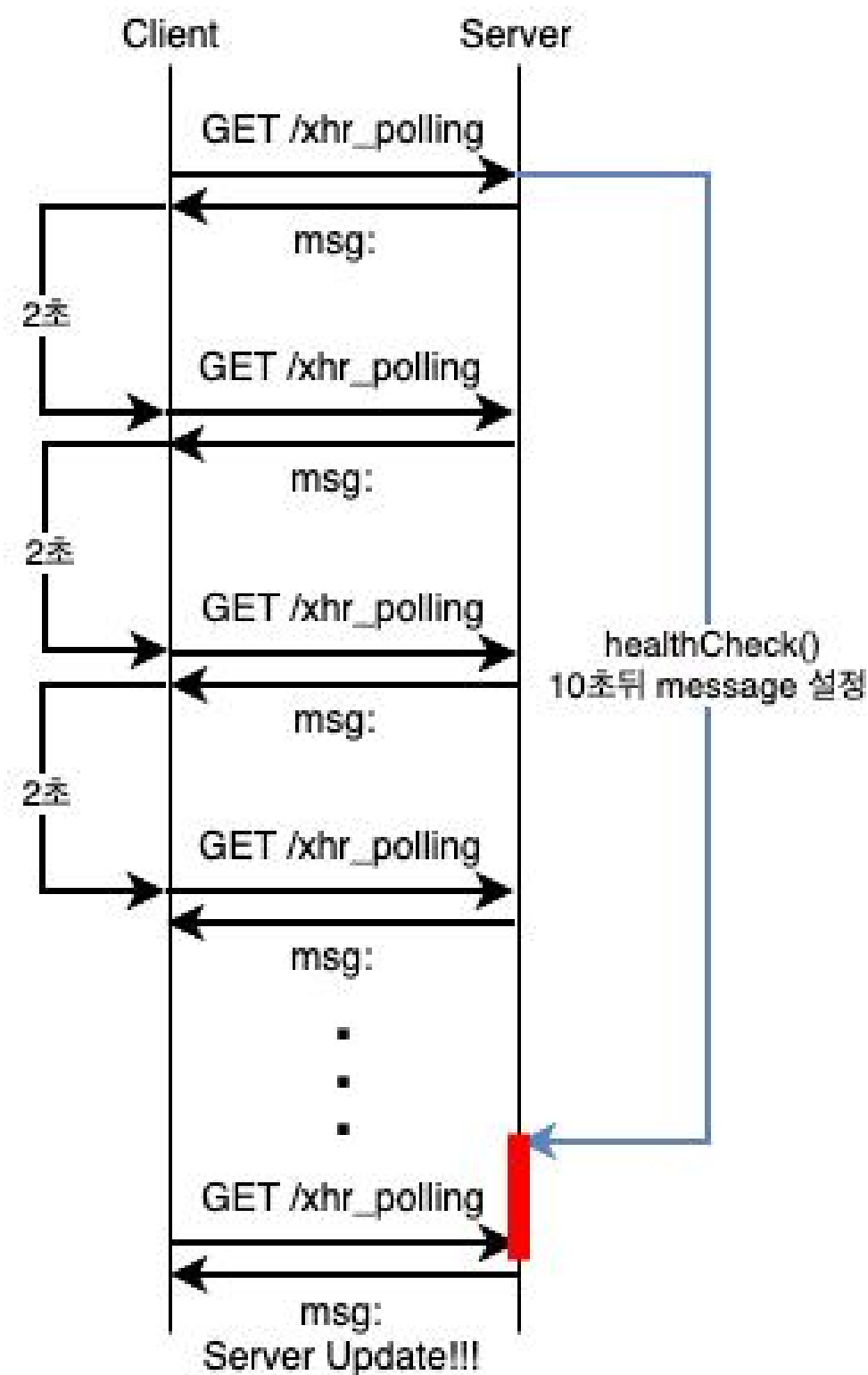


HTTP의 한계 — 왜 실시간이 어려운가?

- HTTP는 클라이언트가 요청해야만 서버가 응답할 수 있다
- 응답이 오면 연결이 끊긴다 → 매번 새로 연결해야 함
- 즉, 서버가 먼저 클라이언트에게 데이터를 보내는 것이 불가능

🌐 WebSocket이 왜 필요한가?

Polling

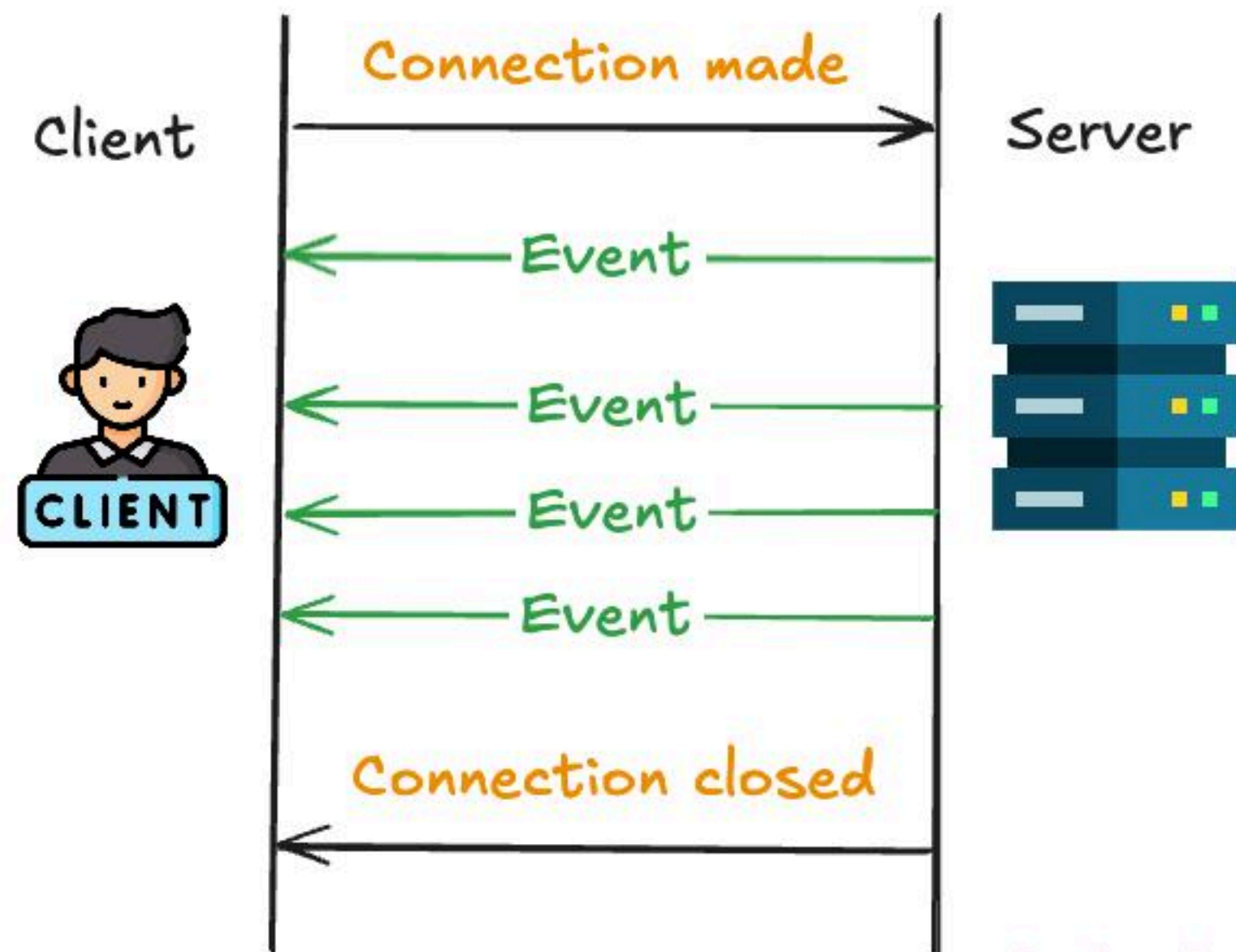


Polling — HTTP로 실시간을 흉내내면?

- HTTP의 한계를 우회하기 위해 일정 주기마다 계속 요청을 보내는 방식
- 실시간처럼 보이지만 사실은 "계속 물어보는 것"
- 사용자가 늘어날수록 서버 요청이 폭발적으로 증가

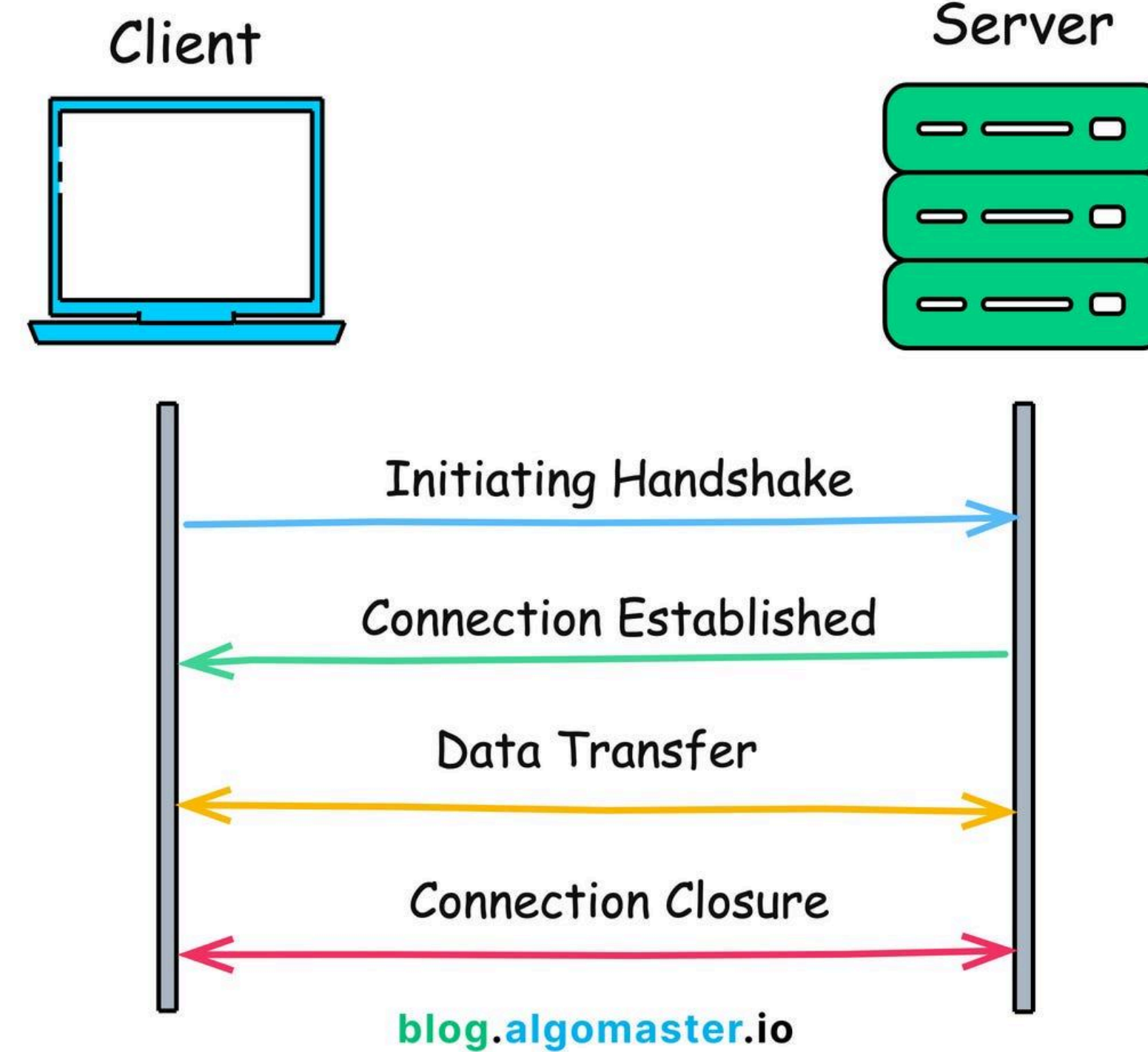
SSE vs WebSocket

Server-Sent Events



By Dhanush Nehru

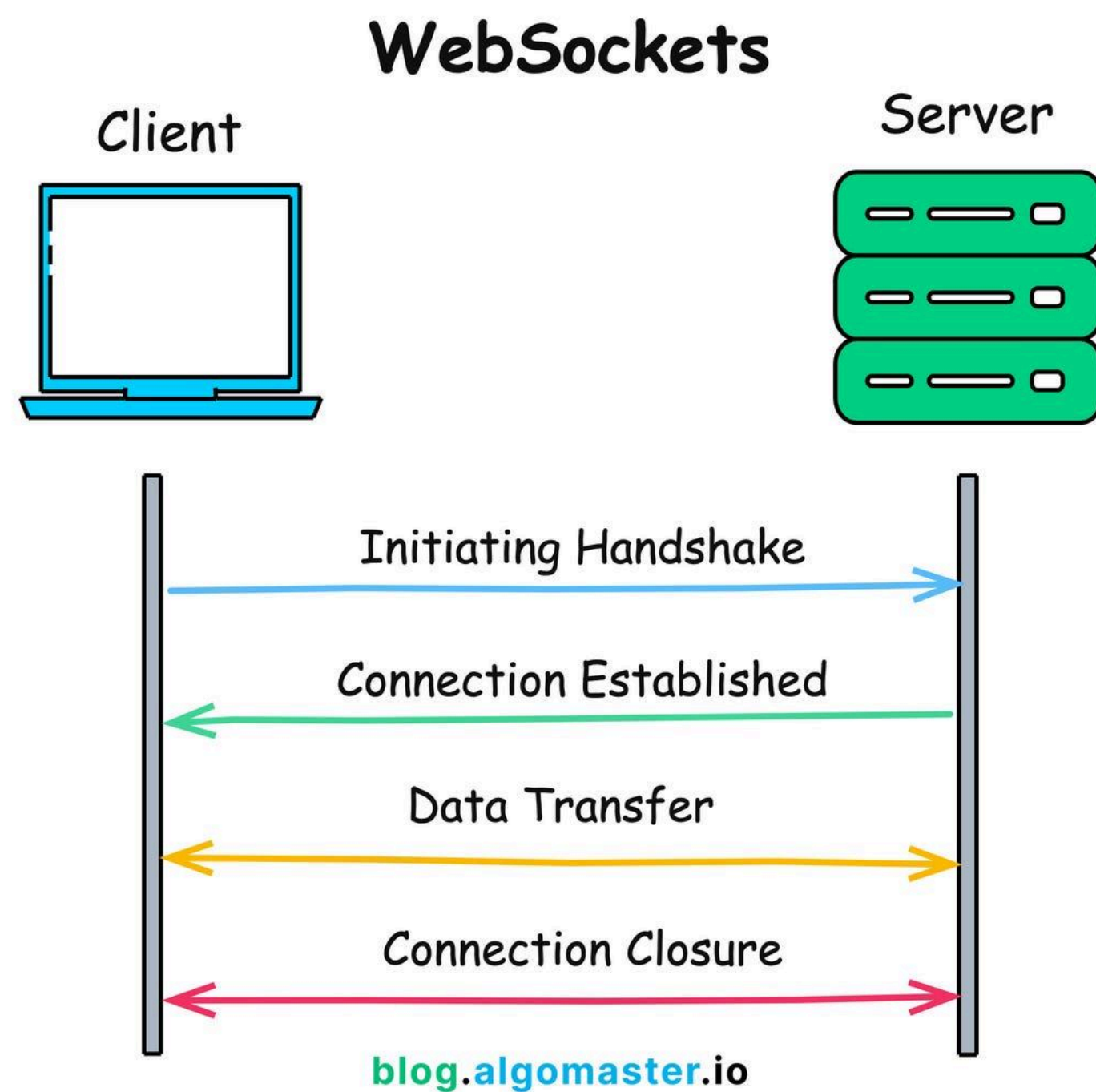
WebSockets



blog.algomaster.io



SSE vs WebSocket



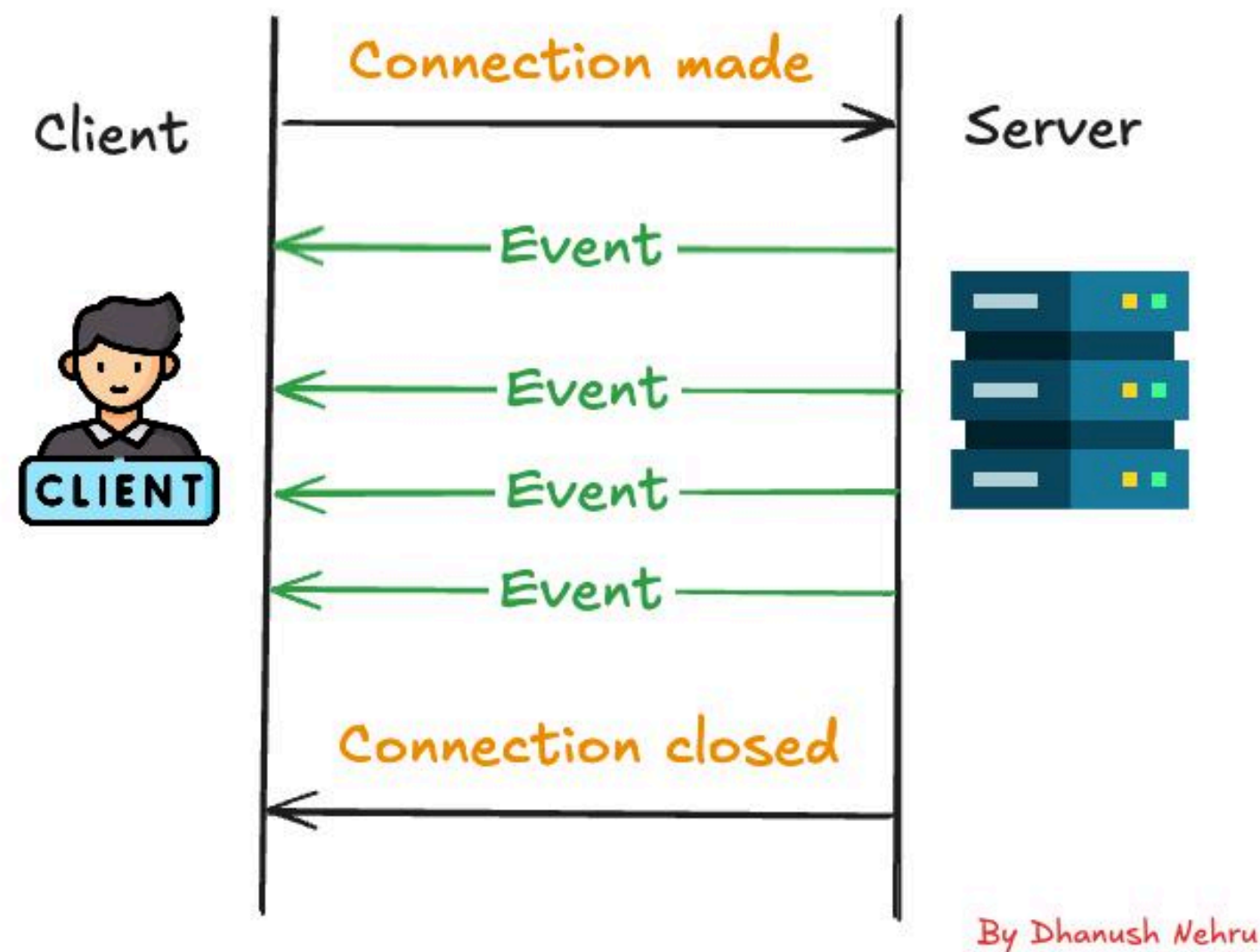
WebSocket이란?

- 한 번 연결하면 연결을 유지하면서 양방향 통신이 가능한 프로토콜
- 서버가 먼저 클라이언트에게 데이터를 보내는 것도 가능
- 연결을 맺는 비용이 한 번뿐이라 Polling보다 훨씬 효율적
- 최초 연결 시 HTTP로 시작해 WebSocket 프로토콜로 업그레이드한다 (핸드셰이크)

예시: 실시간 채팅, 실시간 협업

SSE vs WebSocket

Server-Sent Events



SSE(Server-Sent Events)란?

- 서버 → 클라이언트 단방향 실시간 통신 방식
- HTTP 위에서 동작하기 때문에 별도 프로토콜 없이 구현이 간단
- 연결을 유지하면서 서버가 이벤트를 스트림 형태로 push
- 클라이언트는 연결만 해놓으면 서버가 데이터를 보낼 때마다 자동으로 수신
- 클라이언트가 서버에 뭔가를 보낼 필요가 없는 경우에 적합

예시: 가격 변동 알림, 뉴스피드 업데이트, 배달 상태 알림



SSE vs WebSocket

◆ SSE vs WebSocket

기능	SSE	WebSocket
통신 방식	단방향 (서버 → 클라이언트)	양방향 (서버 ↔ 클라이언트)
프로토콜	HTTP	WebSocket
연결 유지	자동 재연결 지원	수동으로 재연결 필요
사용 예	실시간 알림, 로그 스트리밍	채팅, 게임, 대용량 데이터 전송

SSE는 실시간 알림, 로그 스트리밍 같은 단방향 데이터 전송에 적합하며, WebSocket은 채팅, 게임 같은 양방향 통신이 필요한 경우 더 적합합니다.

선택 기준

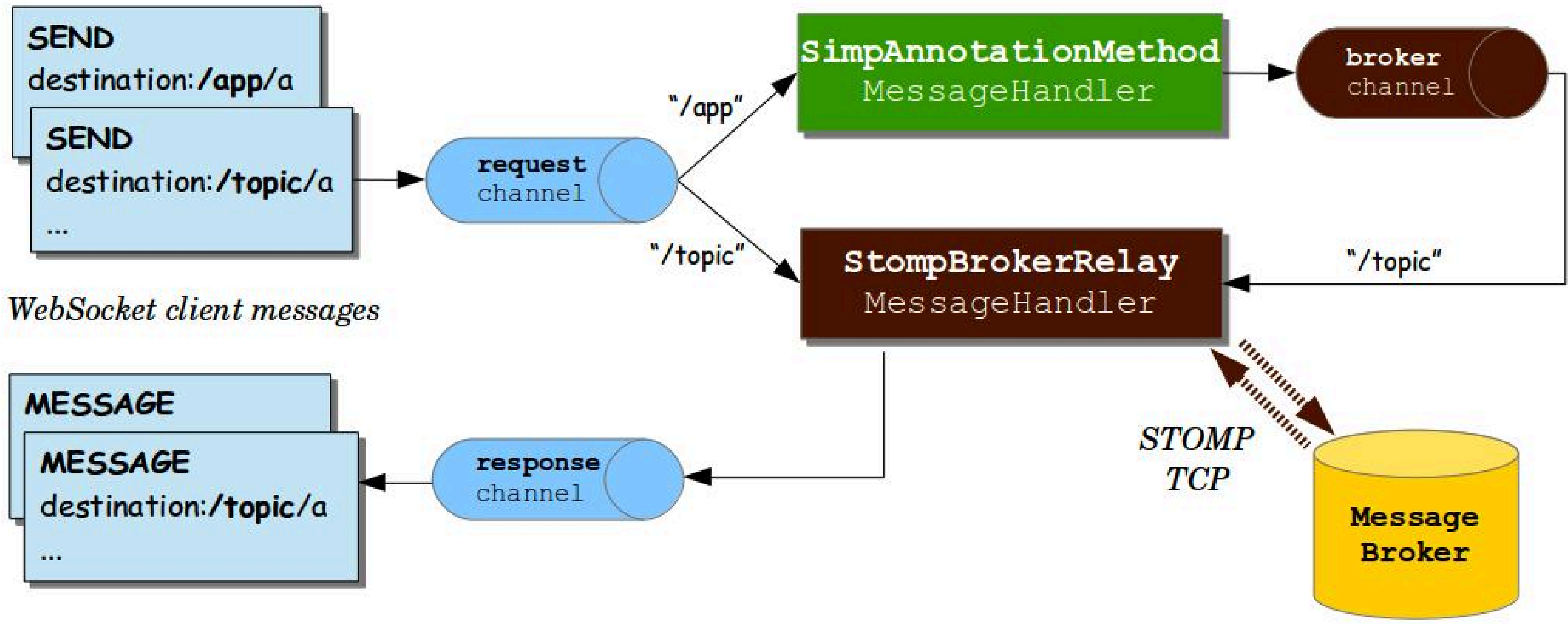
- 서버만 보내면 되는 상황 → SSE
- 클라이언트도 서버에 데이터를 보내야 하는 상황 → WebSocket



순수 WebSocket의 문제

- 순수 WebSocket은 형식이 없는 날것의 문자열 채널
- 메시지만 덩그러니 오기 때문에 누가 보냈는지, 무슨 행동인지, 어디로 가야 하는지 알 수 없음
- 서버가 직접 모든 파싱 로직을 만들어야 한다

STOMP

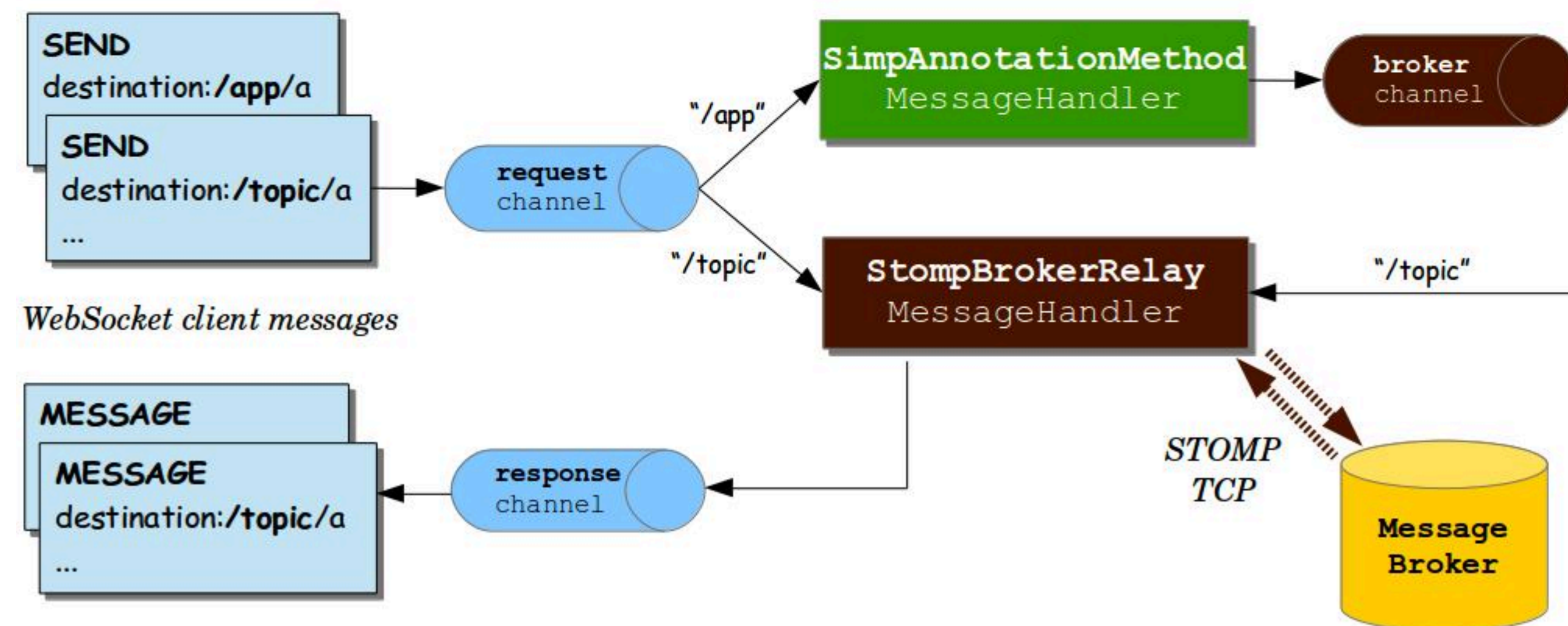




STOMP란?

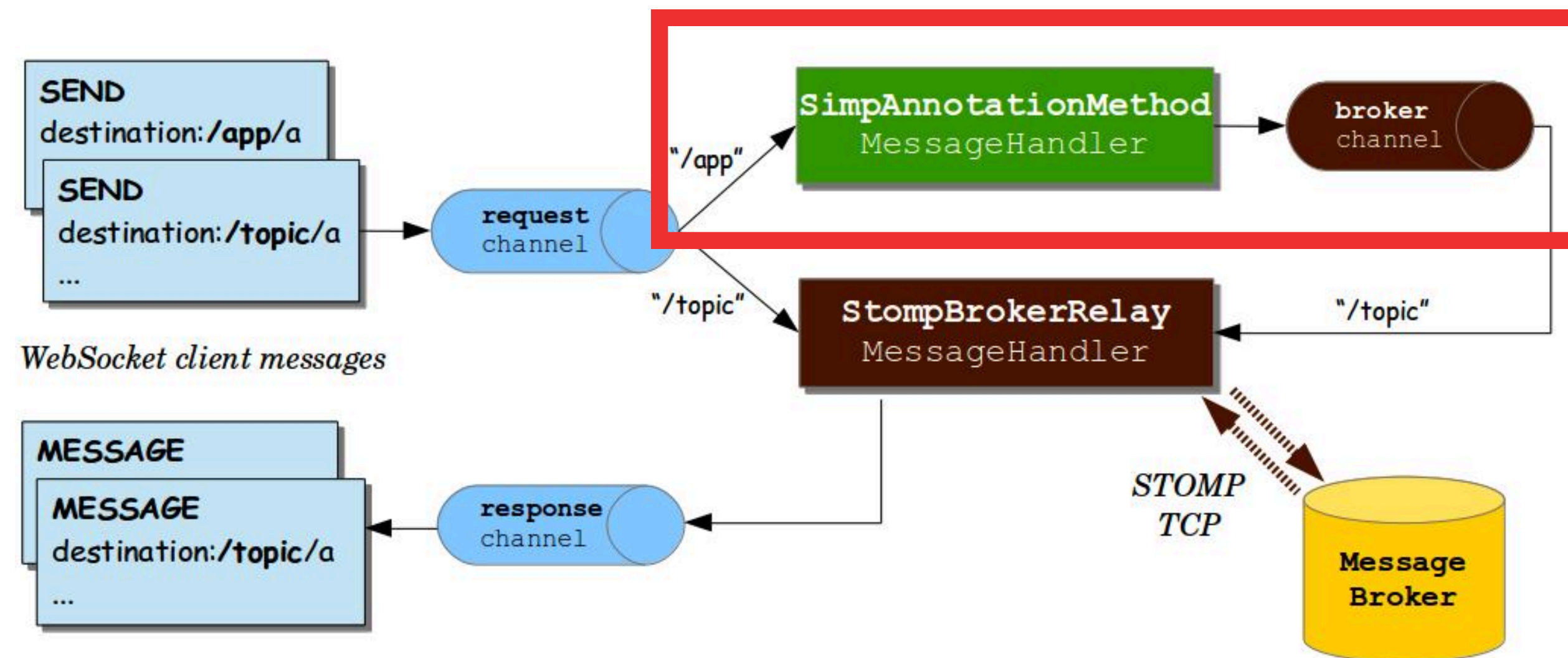
- STOMP(Simple Text Oriented Messaging Protocol)
- WebSocket 위에서 동작하는 메시징 표준 규칙
- 모든 메시지에 목적지(destination)와 명령어가 붙어서 의미가 명확해진다
- STOMP 목적지에는 크게 /app과 /topic이 있다

STOMP — Destination



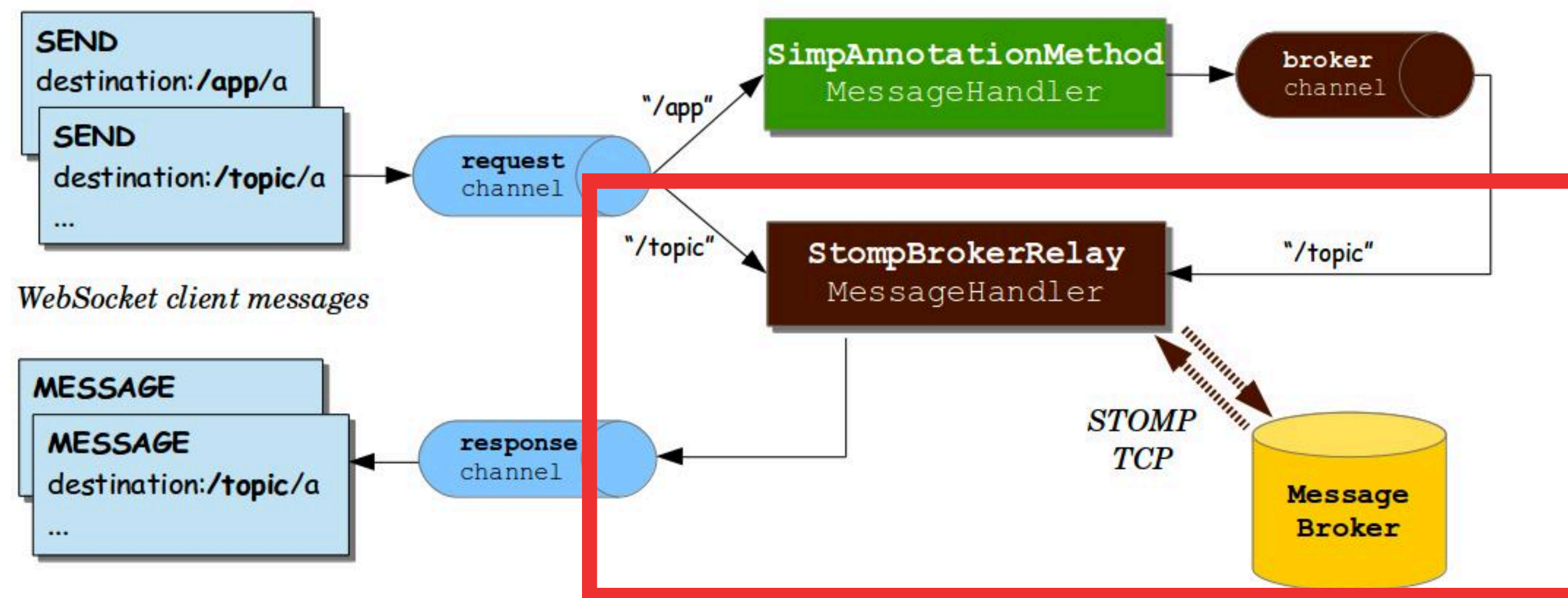
- STOMP에서 모든 메시지는 목적지(Destination)가 붙는다
- 메시지가 어디서 왔고, 어디로 가야 하는지 를 나타내는 주소
- Spring은 이 목적지를 보고 자동으로 적절한 처리를 한다
- 목적지는 크게 /app과 /topic 두 가지로 나뉜다

STOMP — Destination: /app



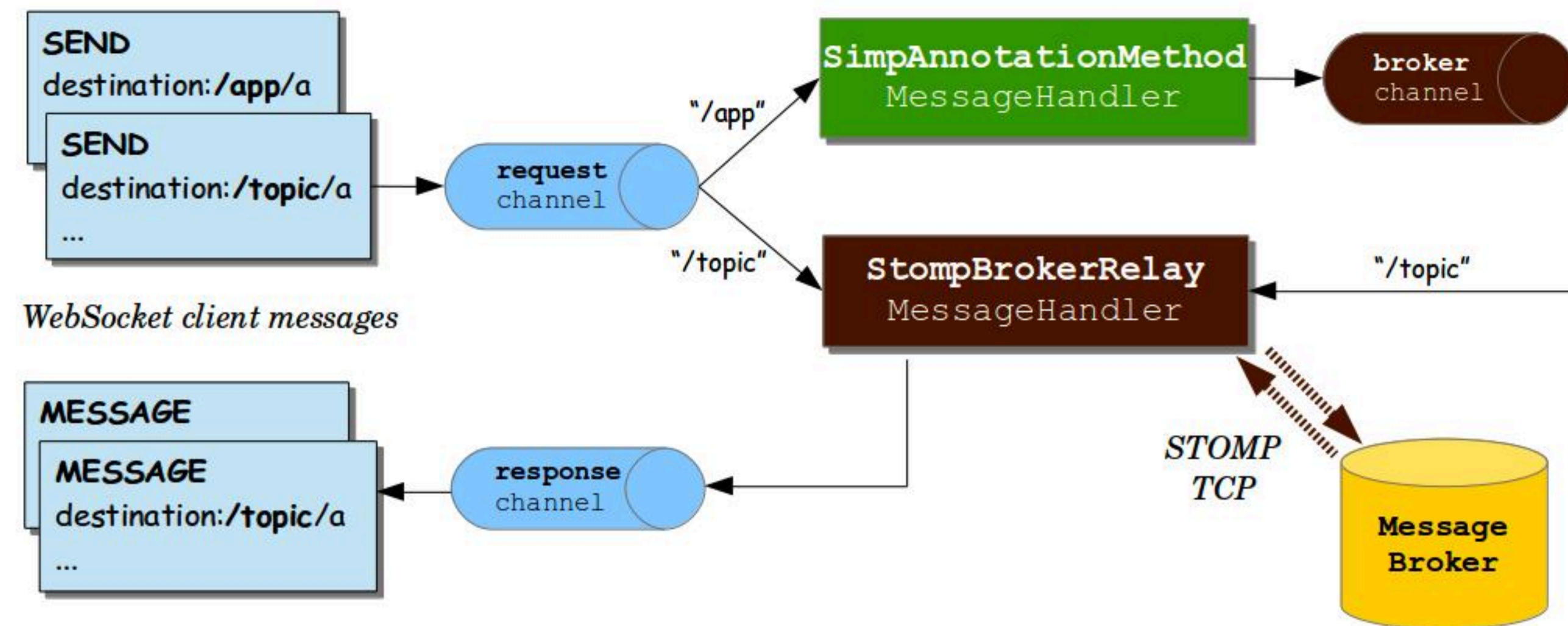
- 클라이언트 → 서버 방향
- 클라이언트가 서버에 처리를 요청할 때 사용
- 프론트엔드는 `publish()`로 전송, 백엔드는 `@MessageMapping`으로 수신

STOMP — Destination: /topic



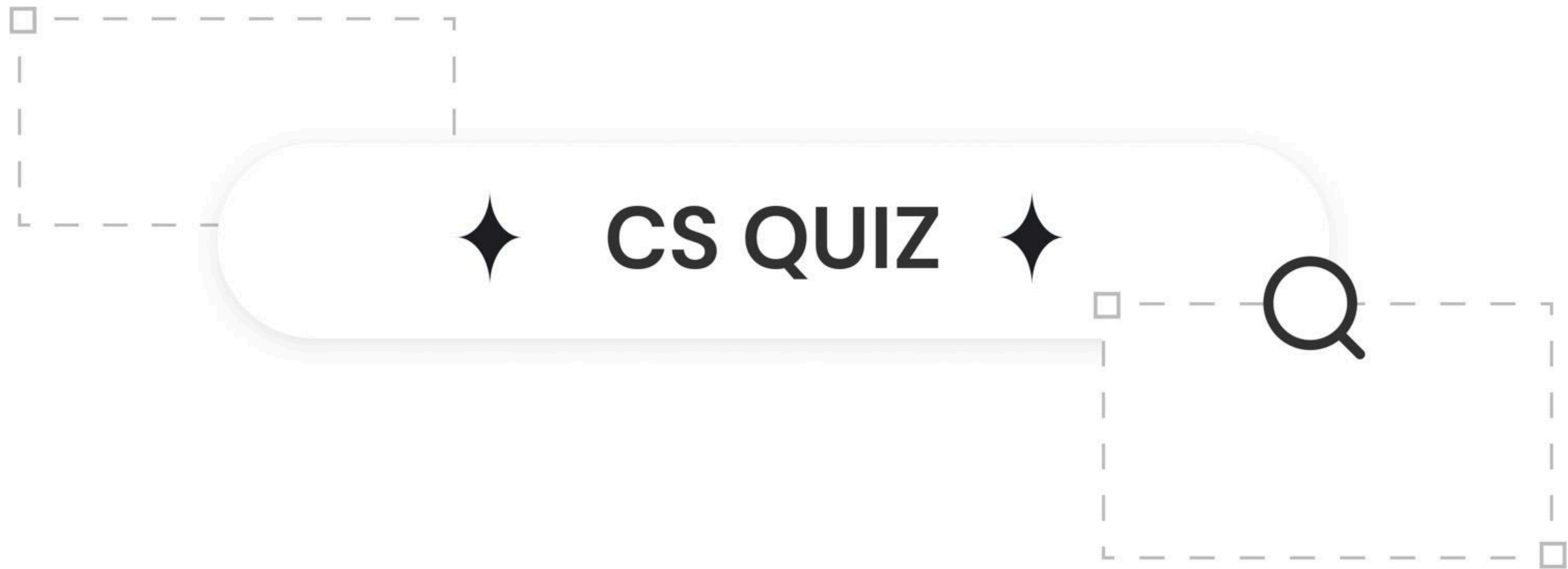
- 서버 → 클라이언트들 방향
- 서버가 구독자 전원에게 동시에 전달할 때 사용 (브로드캐스트)
- 프론트엔드는 `subscribe()`로 구독, 백엔드는 `convertAndSend()`로 전송

STOMP — /app, /topic



정리

- STOMP를 사용하면 /app과 /topic처럼 목적지에 따라 처리 흐름을 명확하게 나눌 수 있다.
- WebSocket을 단순한 문자열 통신이 아닌 구조적인 메시지 기반 통신으로 사용할 수 있게 된다.



Q1

HTTP의 특징으로 옳은 것은?

1. 서버가 먼저 클라이언트에게 데이터를 보낼 수 있다
2. 한 번 연결되면 계속 유지된다
3. 클라이언트의 요청이 있어야 서버가 응답할 수 있다
4. 양방향 통신이 기본적으로 가능하다



A1

HTTP의 특징으로 옳은 것은?

1. 서버가 먼저 클라이언트에게 데이터를 보낼 수 있다
2. 한 번 연결되면 계속 유지된다
3. 클라이언트의 요청이 있어야 서버가 응답할 수 있다
4. 양방향 통신이 기본적으로 가능하다

HTTP는 클라이언트가 요청을 보내야만 서버가 응답할 수 있는 구조입니다. 응답이 오면 연결이 끊기기 때문에 서버가 먼저 데이터를 보내거나 연결을 유지하는 것이 불가능합니다.



Q2

HTTP 통신의 한계를 극복하기 위해 일정 주기마다 반복적으로 요청을 보내 업데이트 여부를 확인하는 방식을 ()(이)라고 합니다.



A2

HTTP 통신의 한계를 극복하기 위해 일정 주기마다 반복적으로 요청을 보내 업데이트 여부를 확인하는 방식을 ()(이)라고 합니다.

Polling

*Polling*은 *HTTP*의 한계를 우회하기 위해 일정 주기마다 반복적으로 요청을 보내는 방식입니다. 사용자가 늘어날수록 서버 요청이 폭발적으로 증가한다는 단점이 있습니다.



Q3

SSE와 WebSocket에 대해 옳은 것은?

1. SSE와 WebSocket 모두 양방향 통신이다
2. SSE는 Server-Sent Events의 약자다
3. WebSocket은 서버에서만 데이터를 보낼 수 있다
4. SSE는 별도의 프로토콜(ws://)을 사용한다



A3

SSE와 WebSocket에 대해 옳은 것은?

1. SSE와 WebSocket 모두 양방향 통신이다
2. SSE는 Server-Sent Events의 약자다
3. WebSocket은 서버에서만 데이터를 보낼 수 있다
4. SSE는 별도의 프로토콜(ws://)을 사용한다

SSE는 Server-Sent Events의 약자입니다. 1번은 틀렸고 SSE는 단방향, WebSocket은 양방향입니다. 3번은 틀렸고 서버에서만 보낼 수 있는 건 SSE입니다. 4번은 틀렸고 ws://는 WebSocket의 프로토콜이며 SSE는 HTTP를 사용합니다.

Q4

STOMP의 /topic 목적지를 이용한 브로드캐스트 구현 시, 프론트엔드와 백엔드에서 각각 사용하는 메서드가 올바르게 짝지어진 것은?

1. 프론트엔드: publish() / 백엔드: @MessageMapping
2. 프론트엔드: subscribe() / 백엔드: convertAndSend()
3. 프론트엔드: publish() / 백엔드: convertAndSend()
4. 프론트엔드: subscribe() / 백엔드: @MessageMapping



A4

STOMP의 /topic 목적지를 이용한 브로드캐스트 구현 시, 프론트엔드와 백엔드에서 각각 사용하는 메서드가 올바르게 짝지어진 것은?

1. 프론트엔드: publish() / 백엔드: @MessageMapping
2. **프론트엔드: subscribe() / 백엔드: convertAndSend()**
3. 프론트엔드: publish() / 백엔드: convertAndSend()
4. 프론트엔드: subscribe() / 백엔드: @MessageMapping

/topic은 브로드캐스트에 사용됩니다. 프론트엔드는 *subscribe()*로 구독하고, 백엔드는 *convertAndSend()*로 구독자 전원에게 메시지를 전송합니다. *@MessageMapping*은 /app에서 사용하는 어노테이션입니다.



Q5

STOMP의 정의에 대해 올바르게 설명한 선지를 고르세요.

1. WebSocket 위에서 동작하는 메시징 표준 규칙
2. SSE 위에서 동작하는 메시징 표준 규칙
3. HTTP의 한계를 우회하기 위해 일정 주기마다
계속 요청을 보내는 방식
4. 프론트엔드: `subscribe()` / 백엔드: `@MessageMapping`



A5

STOMP의 정의에 대해 올바르게 설명한 선지를 고르세요.

1. WebSocket 위에서 동작하는 메시징 표준 규칙
2. SSE 위에서 동작하는 메시징 표준 규칙
3. HTTP의 한계를 우회하기 위해 일정 주기마다
계속 요청을 보내는 방식
4. 프론트엔드: subscribe() / 백엔드: @MessageMapping

STOMP는 WebSocket 위에서 동작하는 메시징 표준 규칙입니다. 3번은 Polling, 4번은 WebSocket에 대한 설명입니다.



Q6

다음 중 기술 선택 기준으로 가장 적절하지 않은 것은?

1. 배달 앱에서 라이더의 실시간 위치를 서버가 클라이언트에게 일방적으로 쏴줘야 할 때 SSE를 고려할 수 있다
2. 실시간 협업 툴처럼 여러 사용자가 동시에 데이터를 주고받아야 할 때는 WebSocket이 적합하다
3. SSE는 WebSocket과 달리 브라우저가 자동 재연결을 지원하지 않으므로 직접 구현해야 한다
4. WebSocket은 SSE보다 구현 복잡도가 상대적으로 높다



A6

다음 중 기술 선택 기준으로 가장 적절하지 않은 것은?

1. 배달 앱에서 라이더의 실시간 위치를 서버가 클라이언트에게 일방적으로 쏴줘야 할 때 SSE를 고려할 수 있다
2. 실시간 협업 툴처럼 여러 사용자가 동시에 데이터를 주고받아야 할 때는 WebSocket이 적합하다
3. SSE는 WebSocket과 달리 브라우저가 자동 재연결을 지원하지 않으므로 직접 구현해야 한다
4. WebSocket은 SSE보다 구현 복잡도가 상대적으로 높다

브라우저가 자동 재연결을 기본으로 지원하는 건 SSE입니다. 직접 구현해야 하는 건 WebSocket입니다.



Q7

[O/X] WebSocket은 연결을 맺는 비용이 한 번뿐이라 Polling보다 효율적이다.



A7

[O/X] WebSocket은 연결을 맺는 비용이 한 번뿐이라 Polling보다 효율적이다.

정답: O

WebSocket은 한 번 연결하면 연결을 유지하기 때문에 매번 새로 요청을 보내야 하는 Polling보다 훨씬 효율적입니다.



Q8

다음은 프로젝트에 STOMP를 적용하기 위해 대화를 나누고 있는 수진이와 다혜의 대화입니다. 각 빈칸에 들어갈 알맞은 목적지를 [보기]에서 골라주세요.

[보기]
/app /topic

수진: 사용자가 보낸 채팅 메시지를 일단 서버가 받아서 DB에 저장하거나 가공하는 '처리'를 요청하려고 해.

이때 클라이언트는 메시지를 어느 경로로 보내야 할까?

다혜: 서버에서 처리한 결과를 여러 사용자(구독자)에게 전달하려면, 어떤 경로를 써야 할까?



A8

다음은 프로젝트에 STOMP를 적용하기 위해 대화를 나누고 있는 수진이와 다혜의 대화입니다. 각 빈칸에 들어갈 알맞은 목적지를 [보기]에서 골라주세요.

[보기]
/app /topic

수진: 사용자가 보낸 채팅 메시지를 일단 서버가 받아서 DB에 저장하거나 가공하는 '처리'를 요청하려고 해.

이때 클라이언트는 메시지를 어느 경로로 보내야 할까?

다혜: 서버에서 처리한 결과를 여러 사용자(구독자)에게 전달하려면, 어떤 경로를 써야 할까?

정답: 수진 - [/app] / 다혜 - [/topic]



Q9

순수 WebSocket만 사용했을 때 발생하는 문제점으로 가장 올바른 것은?

1. 서버가 클라이언트에게 먼저 데이터를 보낼 수가 없다
2. 메시지가 날것의 문자열 형태라 누가, 어디로 보내는 것인지 서버가 알기 어렵다
3. 브라우저가 자동 재연결을 지원하기 때문에 별도의 구현이 필요 없다
4. HTTP 위에서 동작하기 때문에 구현이 너무 단순하다



A9

순수 WebSocket만 사용했을 때 발생하는 문제점으로 가장 올바른 것은?

1. 서버가 클라이언트에게 먼저 데이터를 보낼 수가 없다
2. 메시지가 날것의 문자열 형태라 누가, 어디로 보내는 것인지 서버가 알기 어렵다
3. 브라우저가 자동 재연결을 지원하기 때문에 별도의 구현이 필요 없다
4. HTTP 위에서 동작하기 때문에 구현이 너무 단순하다

순수 *WebSocket*은 형식이 없는 날것의 문자열 채널이라 누가 보냈는지, 무슨 행동인지, 어디로 가야 하는지 알 수 없습니다. 1번은 *HTTP*의 문제점이고, 3번과 4번은 *SSE*에 대한 설명입니다.



Q10



다음은 STOMP에서 /topic으로 메시지를 보낼 때의 데이터 흐름입니다. 올바른 순서대로 나열하세요.

- A. 메시지가 Message Broker에 전달된다
- B. 클라이언트가 /topic/.로 메시지를 전송한다
- C. 해당 토픽을 구독하고 있는 모든 클라이언트들에게 메시지가 전달된다



Q10



다음은 STOMP에서 /topic으로 메시지를 보낼 때의 데이터 흐름입니다. 올바른 순서대로 나열하세요.

- A. 메시지가 Message Broker에 전달된다
- B. 클라이언트가 /topic/.. 으로 메시지를 전송한다
- C. 해당 토픽을 구독하고 있는 모든 클라이언트들에게 메시지가 전달된다

정답: B - A - C

클라이언트가 /topic으로 메시지를 전송하면(B), Message Broker에 전달되고(A), 해당 토픽을 구독하고 있는 모든 클라이언트들에게 브로드캐스트됩니다(C).

